DAY 14 - Service Mesh, Anthos & End-to-End Microservices
- Advanced Load Balancing & Networking

Advanced Load Balancing & Networking
- Multi-service routing
- Internal vs external service exposure
- Network security considerations

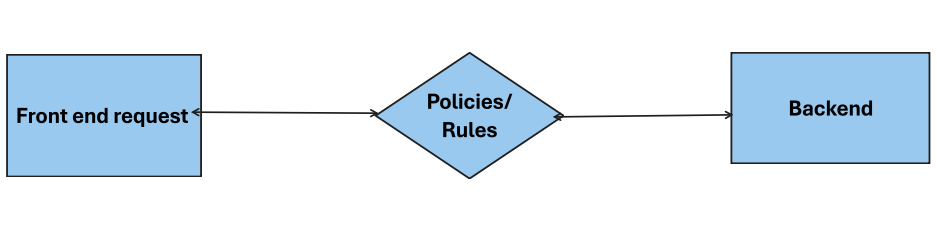
Hands-On Lab:
- Configure advanced routing for microservices

Validate secure communication

* Multi-Service Routing:

Multi-service routing is about sending incoming requests to the correct backend service based on rules. It's a key part of microservices architectures.

It's a core concept in microservices architecture, usually handled by:
- API Gateways
- Load balancers
- Service meshes



1. Path-Based Routing 2. Host-Based Routing 3. Header-Based Routing 4. Canary / A/B Routing

1. Path-Based Routing:

Route requests based on the URL path.

Example:

/api/users → User Service
/auth/login → Auth Service

* When to use:

REST APIs
Single domain, multiple services

* Why it's popular:
Simple
Easy to implement
Works well with frontend + backend split

* Why Path-Based Routing?

Path-based routing is popular because it solves a very practical problem:

How do I split one application into multiple services without changing everything?

1. * Keeps a Single Domain:
You don't need multiple domains.

Instead of:
api.example.com
auth.example.com

You can use:

example.com/api
example.com/auth

Easier DNS, simpler architecture.

2. * Minimal Changes for Clients:
Clients (frontend, mobile apps) don't need major updates.
- Same base URL
- Only paths differ

This is huge during migrations.

3. Perfect for Breaking Monoliths
When moving from a monolith → microservices:

Start routing /api/users → new service
Keep rest in monolith

- Gradual migration without downtime.

4. Simple to Implement
Works out-of-the-box with:

- Kubernetes Ingress
- API Gateways
- Reverse proxies

No complex config needed.

5. Natural Fit for REST APIs:
REST APIs are already path-based:
/users
/orders
/payments

Routing aligns naturally with API design.

6. Works Well with Frontend Apps:
Example:
/app → React frontend
/api → Backend

Clean separation without different domains.

7. No Dependency on Headers or Special Logic
Unlike header-based routing:
- No need to modify requests
- No need for custom headers

Path-based routing is popular because it's:

Simple
Non-intrusive
Migration-friendly
Universally supported

2. Host-based routing: Host-based routing = routing based on domain/subdomain.

Basic incoming requests:

api.example.com → API service
admin.example.com → Admin service

Result:
- Requests are routed purely based on the hostname
- No need to inspect:
- URL paths
- Headers

Why HostBased Routing needed?
1. Clear Separation of Services:
Each service gets its own identity:

api.example.com → API
admin.example.com → Admin

Cleaner architecture, easier to reason about.

2. Independent Scaling & Deployment
You can:
- Scale API without touching admin
- Deploy services independently

Common in mature microservices setups.

* Examples: Product Lines / Services:

Requests:
https://blog.example.com
https://shop.example.com
https://forum.example.com

Routing:
blog.example.com → Content Management Service
shop.example.com → E-commerce Service
forum.example.com → Community Forum Service

Use Case: Different products under the same company, each on its own subdomain.

* Regional Routing:
https://us.example.com
https://eu.example.com
https://apac.example.com

Routing:

us.example.com → US region servers
eu.example.com → Europe servers
apac.example.com → Asia-Pacific servers

Use Case: Low latency and data residency compliance by routing to regional services.

* Environment-Based Routing:
https://dev.example.com
https://staging.example.com
https://prod.example.com

Use host-based routing when:

- System is growing
- Teams/services need isolation
- You want clean, production-grade architecture

* Use path-based routing when:
You want simplicity
You're early-stage or migrating

3. Header-Based Routing:

Header-based routing sends requests to different backend services based on HTTP headers, instead of the URL path or domain.
This is commonly used for API versioning, feature flags, and testing.

* How It Works:
Client Request Example:

GET /users
Host: api.example.com
Header: version=v2

Routing Rules:
version=v1 → Old Service
version=v2 → New Service

result: The router reads the header and directs traffic to the appropriate service.

Examples:
1. API Versioning:

Requests:
GET /users
Header: version=v1 → Old API
Header: version=v2 → New API

Use Case:
- Roll out a new API version without changing URLs
- Backward compatibility for old clients

2. Feature Flag / Beta Testing
Requests:
GET /checkout
Header: beta-user=true
Routing:
beta-user=true → Beta checkout service
beta-user=false → Stable checkout service
Use Case: Test new features on a small set of users without affecting everyone

4. Canary or A-B routing
-> Canary or A-B routing is a method of gradually splitting traffic between multiple versions of a service or feature.

v1: Stable version : 80% Traffic
v2: New version or experimental feature: 20% Traffic

Instead of switching all users at once, traffic is routed partially to v2 to test performance, stability, or user acceptance.

* How It Works:

Traffic Split Example:

90% of users → v1 (stable version)
10% of users → v2 (new version)

Mechanism:
- Usually implemented via load balancers, service mesh, or API gateways
- Decisions can be random or based on user attributes (e.g., region, user ID, cookies)

* Examples:

1. Safe Deployment / Release:

v1 → current production
v2 → new release candidate
10% of traffic goes to v2 for testing
If v2 works fine → gradually increase traffic
If v2 fails → rollback without affecting most users

2. A/B Testing / Feature Experiments:
v1 → old UI or old checkout flow
v2 → new UI / feature
10% of users see the new UI → test metrics like engagement or conversion
Decide based on data whether to release to 100%

3. Regional Rollouts:

-> Users from Europe → v1
-> Users from Asia → v2

Test regional infrastructure or compliance requirements

4. Internal Testing

Employees or beta users → v2
General public → v1

Safe way to test features internally without affecting all customers(Public)

Advantages:
- Reduced risk: Only a small fraction of users experience new changes
- Quick rollback: If v2 fails, traffic can be routed back to v1 instantly
- Data-driven decisions: Compare v1 vs v2 metrics before full rollout

* Disadvantages:
- Slightly complex setup (requires gateway, service mesh, or load balancer)
- May require user tracking to ensure consistent experience
- Monitoring and metrics collection are crucial

* Real-World Example:
Imagine an e-commerce checkout system:

Traffic	Version	Users
90%	v1	All normal
10%	v2	Beta testers

If 10% users complete checkout successfully → gradually increase v2 traffic
If errors occur → rollback immediately

*** Internal vs external service exposure:

Service exposure determines who can access a microservice or API: only internal components, or external users as well.

1. Internal Services:

Means :
- Accessible only within the cluster or internal network
- Not reachable by public internet

* Examples:
- Authentication service
- Database service
- Payment processing service

* Benefits:
- Security: Less risk of public attacks
- Performance: Internal network is faster
- Controlled access: Only trusted components can call the service

How to Implement:

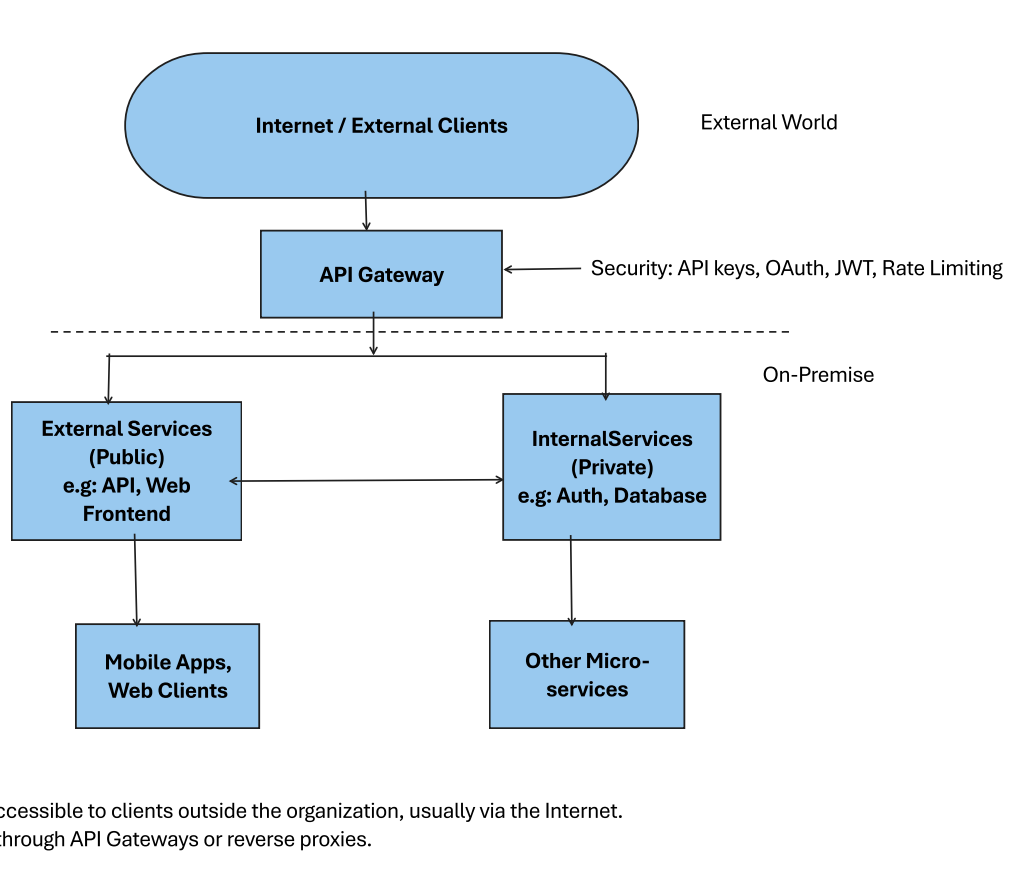
- Kubernetes ClusterIP services
- Service mesh with internal routing
- Firewall / network policies

Simple terms: Service is only accessible within the organization's network or the internal microservices ecosystem.
- Used for backend services, databases, or microservices that support other services.

Authentication Service → accessed only by API Gateway
Payment Processing Service → internal microservices call it, not external clients
Inventory Service → used by order management service, not directly by customers

Typical Setup:
- Service Mesh (e.g., Istio, Linkerd)
- Internal load balancer
- Private subnets in cloud

** External Service Exposure:
External services are accessible to clients outside the organization, usually via the Internet.
They often sit behind API Gateways or reverse proxies for management and security.


Internet / External Clients
External World
API Gateway
Security: API keys, OAuth, JWT, Rate Limiting
On-Premise
External Services (Public) e.g: API, Web Frontend
InternalServices (Private) e.g: Auth, Database
Mobile Apps, Web Clients
Other Micro-services

External Clients → Access services via the API Gateway.

API Gateway → Handles security, routing, rate limiting.

External Services → Publicly accessible services for users/partners.

Internal Services → Only accessible inside the cluster/network. [On-Premise]

Internal services can communicate with external services but are not directly exposed to the internet.

- Service is accessible to clients outside the organization, usually via the Internet.
- Often goes through API Gateways or reverse proxies.

Examples:

api.example.com/users → mobile apps and web clients call this
auth.example.com/login → external clients authenticate here
SaaS platform APIs → accessible by customers or partners
PaaS - > development

* Benefits:
- Makes services usable by customers, partners, or third-party apps
- Enables integrations and public APIs

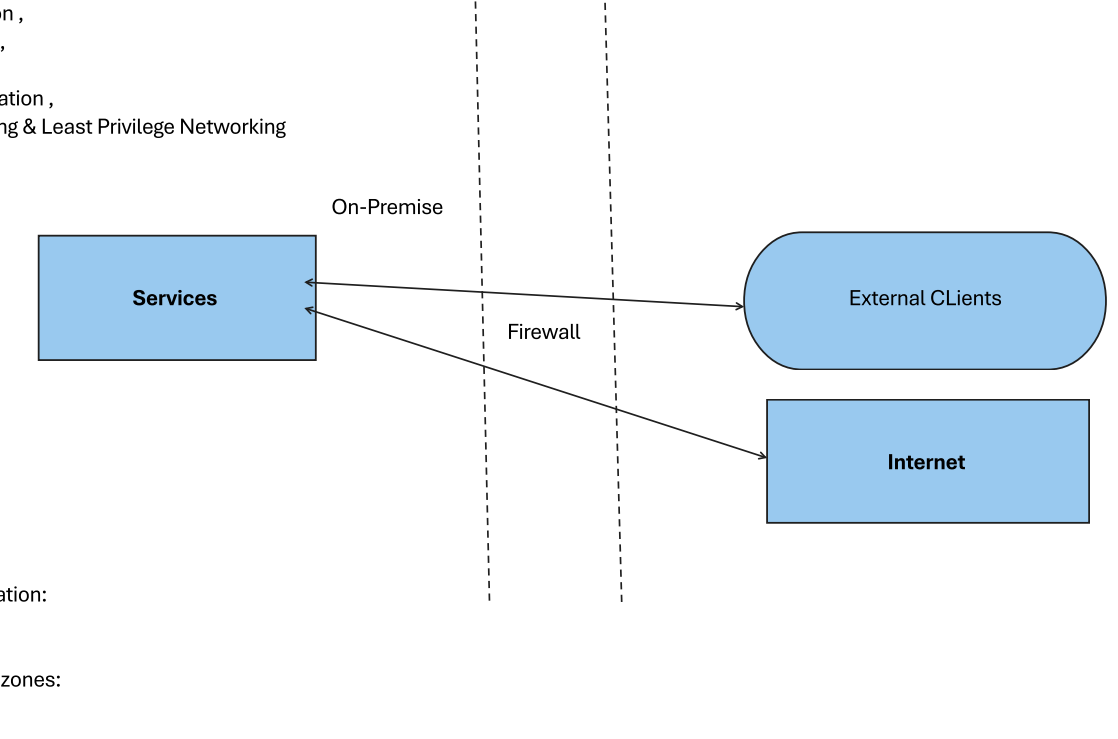
Typical Setup:
- Public load balancer
- API Gateway
- Security: API keys, OAuth, JWT (Json based Token), rate limiting

Feature	Internal Service	External Service
Access	Only internal network / services	Public Internet / clients
Security	Easier, lower attack surface	Requires authentication, monitoring
Purpose	Backend, support, core logic	User-facing APIs, integrations
Examples	Inventory Service, Payment API	Public API, Auth Service

* Network security considerations:

Network security is about protecting communication between services and external clients while minimizing exposure to threats.

Key areas:
Segmentation and Isolation ,
Service Exposure Control ,
Encryption in Transit ,
Authentication & Authorization ,
Traffic Filtering & Monitoring & Least Privilege Networking



* 1. Segmentation & Isolation:

What it means:
Divide your network into zones:
- Internal vs external
- Dev vs Prod

Isolate sensitive services from public access

* Example:
- Internal services reside in private subnets
- Public services sit behind a load balancer

* Benefit:
- Limits the blast radius if a service is compromised
- Prevents attackers from easily moving laterally within the network

- Limit which services can communicate to only what is necessary.
Techniques:
- Namespaces: Logical separation of workloads.
- Network Policies (Kubernetes): Allow or deny traffic between pods.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: allow-frontend-to-backend
spec:
 podSelector:
  matchLabels:
   app: backend
 ingress:
 - from:
  - podSelector:
    matchLabels:
     app: frontend
```

VPCs / Subnets: In GCP, keep sensitive services in private subnets.

Benefit: Even if one pod is compromised, the attacker can't freely reach other services.

Case Scenario: Healthcare SaaS Platform:

ABC CorpCompany: GCP Cloud-based platform providing medical record management and telemedicine services.

Services:

- Patient Portal (Web & Mobile) – patients book appointments, view records
- API Service – integrates with partner hospitals and insurance providers
- Medical Records Service – stores sensitive patient health data
- Admin Dashboard – internal staff management

* Challenge:
- Protect sensitive patient data from public exposure
- Prevent breaches in one service from affecting others
- Separate environments for development, staging, and production

* Segmentation & Isolation Implementation:

1. Network Zones*

--> Public zone: Patient portal and partner-facing APIs

--> Private zone: Medical records service, admin dashboard

--> Dev/test zones: Fully isolated from production

2. Access Control

--> API service can access medical records, but only via authenticated and encrypted channels

--> Admin dashboard accessible only from internal network via VPN

3. Subnet & Firewall Rules:

- Public services in public subnets behind load balancer
- Private services in private subnets, no direct internet access
- Security groups limit access to only required ports (HTTPS, API ports)

4. Service Mesh / Internal Routing:
--> Internal services communicate via mTLS
--> Monitoring & logging - detect unauthorized attempts to access sensitive data

** Benefits Achieved :

- Sensitive medical records protected → even if patient portal is compromised, backend data remains secure
- Isolated dev/test environments → prevents accidental leaks of real patient data
- Blast radius minimized → attacks contained within specific zones
- Regulatory compliance → HIPAA and other healthcare regulations easier to enforce

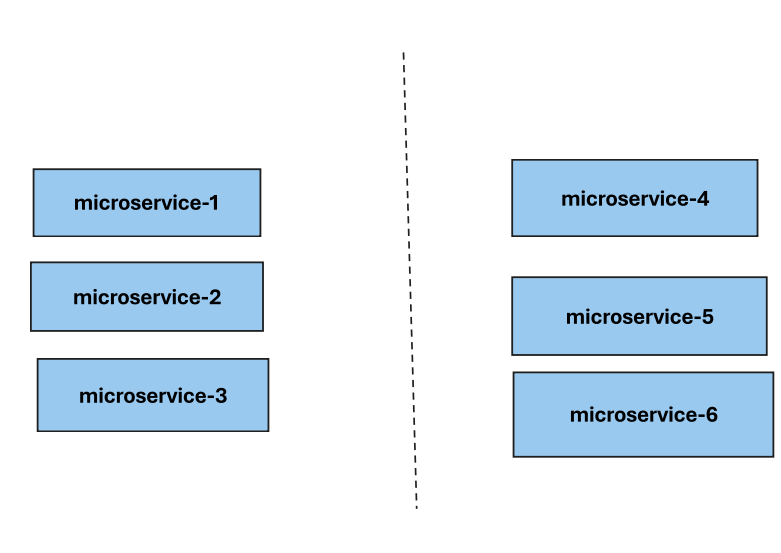
Real-World Parallel:

Healthcare SaaS companies like Epic Systems or Cerner isolate sensitive patient data internally while exposing only necessary services to patients and partners.

2. Service Exposure Control:

Service exposure control determines which microservices are accessible internally (CLuster / VPC) versus those that can be accessed externally.

Type	Access Scope	Kubernetes Example
Internal	Only accessible **within the cluster or VPC**	ClusterIP, Internal LoadBalancer
External	Accessible **from the Internet**	LoadBalancer, Ingress



It's a critical part of microservices architecture, cloud deployments, and security design. Proper exposure control helps minimize attack surfaces and secure sensitive services.

Internal services → only talk to other microservices; hidden from outside world
External services → exposed via proper gateways or ingress controllers

3. Encryption in Transit:

Encryption in transit ensures that all communication between clients and services, or between services themselves, is protected from interception or tampering.

* TLS / HTTPS:

All traffic between clients and services should be encrypted using TLS/HTTPS.

In Kubernetes:

Configure TLS certificates in Ingress
Use sidecar proxies (like Envoy) for TLS termination

* Mutual TLS (mTLS)
- Both client and server verify each other's identity
- Commonly used in service meshes:

Istio
Linkerd

* Benefit:
- Ensures only authorized services communicate
- Adds strong identity verification for internal microservices

Key Insight:

- TLS/HTTPS = encrypts client ↔ service traffic
- mTLS = encrypts service ↔ service traffic + identity verification
- Together, they prevent data leaks, tampering, and unauthorized access

** 4. Authentication & Authorization :

Authentication and authorization determine who can access your services and what actions they can perform.

* Authentication:

Verifies the identity of users or services.
Common methods:
API Keys → Simple token-based access
OAuth Tokens → Standard for user authentication and delegated access
JWTs (JSON Web Tokens) → Encodes identity and permissions securely

Purpose: Ensure that only authorized clients or services can connect.

* Authorization:

- Determines what authenticated users or services are allowed to do.
- Use Role-Based Access Control (RBAC):
- Define roles and permissions
- Example: read-only, admin, billing-service-access

Purpose: Prevent unauthorized actions even from authenticated users/services.

Centralized Control:

*Ingress or API Gateway:
Handles authentication for incoming traffic
Applies rate limiting, throttling, and logging
Ensures consistent security across multiple services
* Service Mesh (optional):
Can enforce service-to-service authentication and authorization internally

* Key Insight
Authentication = confirms identity
Authorization = enforces what identities can do
Together, they protect services from unauthorized access and misuse

5. Traffic Filtering & Monitoring

Control and observe network traffic to protect services from unauthorized access, abuse, or attacks.

Firewall Rules / Security Groups:

- Restrict which IP ranges or sources can access services
- Example: Only allow office VPN or partner IPs to access internal APIs

DDoS Protection: [Distributed Denial of Service]
- DDoS (Distributed Denial of Service) is when attackers overwhelm your service with massive traffic to make it unavailable.
Protection involves preventing, mitigating, and monitoring such attacks

Protect public endpoints from large-scale attacks
Cloud solutions:
GCP Cloud Armor
AWS Shield

* Logging & Monitoring:

Collect network activity to detect suspicious behavior
Examples:
- VPC Flow Logs → track internal/external traffic
- Cloud Logging / Stackdriver → monitor anomalies

* Rate Limiting:

Prevent service overload or abuse
Can be implemented at:
- API Gateway / Ingress
- Service mesh sidecars

Benefit: Ensures service availability even under high traffic

* Least Privilege Networking :

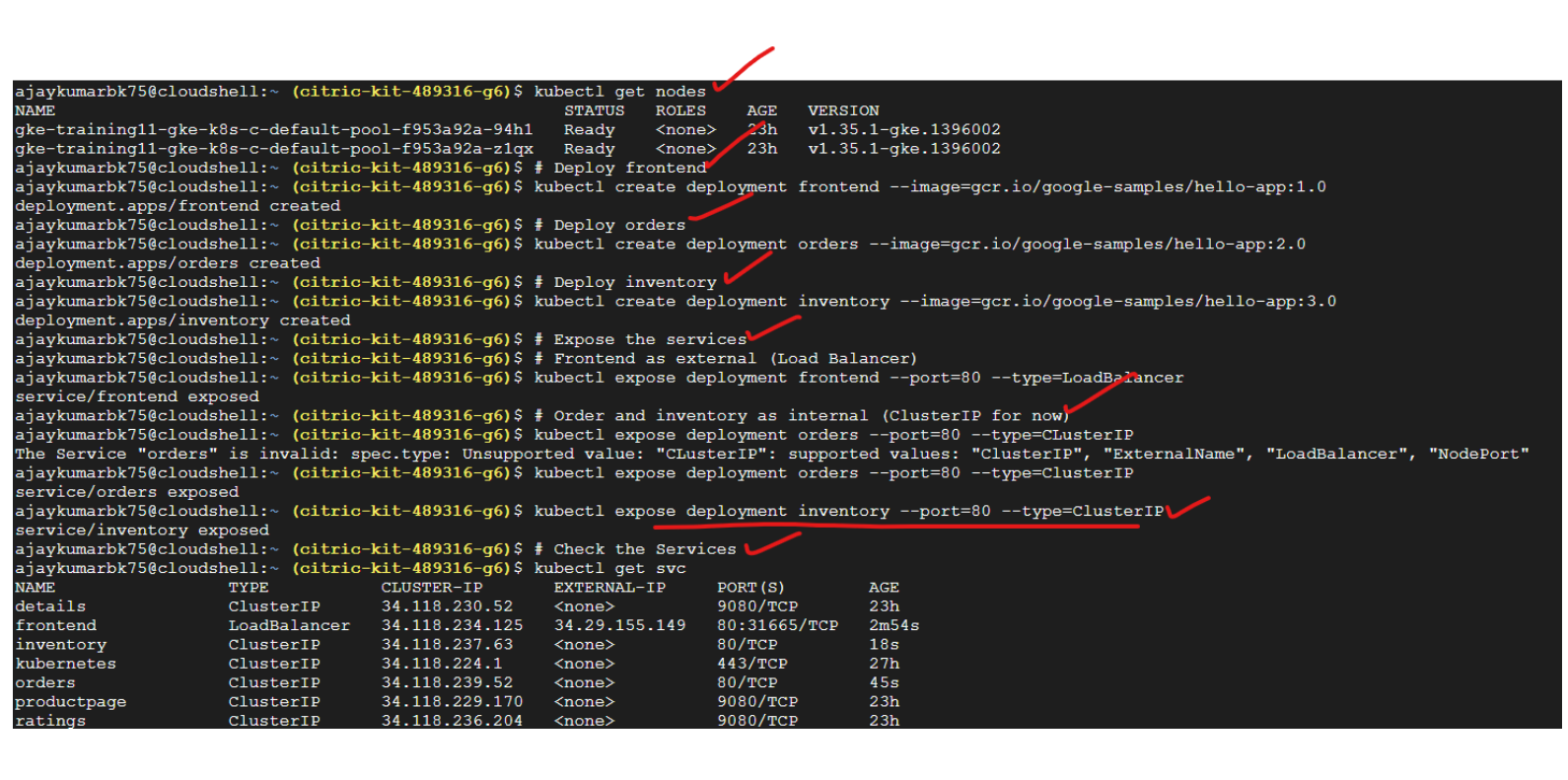
Only allow necessary communication between services and open only required ports.

Best Practices:

Port restrictions:
Open only the ports needed for the service to function
Example: Web service → 443, Database → 5432
Service-to-service restrictions:
Database pods accept traffic only from backend pods
Frontend or public traffic cannot reach internal databases
Policy enforcement:
Use Kubernetes NetworkPolicies
Service meshes can enforce internal traffic rules

Benefit: Minimizes attack surface and prevents lateral movement if a service is compromised

Key Insight:
Traffic filtering + monitoring → Detect and block unwanted traffic
Least privilege networking → Ensure services only communicate as necessary
Together, these practices harden microservices networks against attacks and misconfigurations



```
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl get nodes
NAME                                                  STATUS   ROLES    AGE   VERSION
gke-training11-gke-k8s-c-default-pool-f953a92a-94h1   Ready    <none>   23h   v1.35.1-gke.1396002
gke-training11-gke-k8s-c-default-pool-f953a92a-z1qx   Ready    <none>   23h   v1.35.1-gke.1396002
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ # Deploy frontend
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl create deployment frontend --image=gcr.io/google-samples/hello-app:1.0
deployment.apps/frontend created
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ # Deploy orders
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl create deployment orders --image=gcr.io/google-samples/hello-app:2.0
deployment.apps/orders created
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ # Deploy inventory
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl create deployment inventory --image=gcr.io/google-samples/hello-app:3.0
deployment.apps/inventory created
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ # Expose the services
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ # Frontend as external (Load Balancer)
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl expose deployment frontend --port=80 --type=LoadBalancer
service/frontend exposed
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ # Order and inventory as internal (ClusterIP for now)
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl expose deployment orders --port=80 --type=CLusterIP
The Service "orders" is invalid: spec.type: Unsupported value: "CLusterIP": supported values: "ClusterIP", "ExternalName", "LoadBalancer", "NodePort"
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl expose deployment orders --port=80 --type=ClusterIP
service/orders exposed
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl expose deployment inventory --port=80 --type=ClusterIP
service/inventory exposed
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ # Check the Services
ajaykumarbk75@cloudshell:~ (citric-kit-489316-g6)$ kubectl get svc
NAME          TYPE           CLUSTER-IP       EXTERNAL-IP     PORT(S)        AGE
details       ClusterIP      34.118.230.52    <none>          9080/TCP       23h
frontend      LoadBalancer   34.118.234.125   34.29.155.149   80:31665/TCP   2m54s
inventory     ClusterIP      34.118.237.63    <none>          80/TCP         18s
kubernetes    ClusterIP      34.118.224.1     <none>          443/TCP        27h
orders        ClusterIP      34.118.239.52    <none>          80/TCP         45s
productpage   ClusterIP      34.118.229.170   <none>          9080/TCP       23h
ratings       ClusterIP      34.118.236.204   <none>          9080/TCP       23h
```